

InterLock AI Review Console (Demo Project)

Ingestion · Extraction · Comparison · Citation · Audit Trail

Version 0.1 · May 2026 · InterLock AI / AI Fund Build Challenge

Core design principle: Every AI-generated flag must be grounded in source evidence.

Every extracted claim must be inspectable. The LLM never detects a discrepancy —

it only explains one the comparison engine already found deterministically.

1. Architecture Overview

The pipeline has five sequential stages. Each stage has a defined input schema, output schema, and failure mode. No stage skips forward — the comparison engine cannot run without a populated claim store, and the LLM explanation pass cannot run without a populated discrepancy store.

Stage	Input → Output	Key Constraint
1. Ingestion	PDF bytes → PageEvidence rows	Every page tagged with extraction_method + confidence
2. Extraction	PageEvidence text → ExtractedClaim rows	Regex runs first; LLM only normalizes, never invents
3. Comparison	ExtractedClaim rows → Discrepancy rows	100% deterministic; no LLM in this stage
4. Explanation	Discrepancy + source snippets → explanation	LLM receives verbatim snippets only, no comparison context
5. Report / UI	All tables → review UI + audit export	Every flag must cite claim_a_id; orphan flags forbidden

2. Data Model

All state lives in SQLite (swappable to Postgres). Four tables form the audit chain. Every downstream record points back to an upstream source — no discrepancy exists without a claim, no claim exists without a page, no page exists without a document.

2.1 Document

id · filename · doc_type · uploaded_at · page_count · file_size

doc_type is set by the user at upload: owner_spec | control_spec | vendor_submittal | unknown. The comparison engine uses doc_type to determine which documents define requirements and which are being evaluated against them.

2.2 PageEvidence

id · document_id · page_number · text · extraction_method · confidence

One row per page. extraction_method is one of: text, table, text+table, ocr. confidence is set by the parser: 1.0 for clean text, 0.75 for OCR output. This row is the immutable evidence anchor — every downstream claim references it through document_id + page_number.

2.3 ExtractedClaim

id · document_id · page_number · claim_type · entity · parameter_name

value_raw · value_normalized · unit · condition · source_text · extraction_method · confidence

One row per extracted claim. source_text is the verbatim sentence(s) containing the claim — this is the citation text shown in the UI. entity is a normalized concept key (e.g. generator_rating_type) that allows the comparison engine to group claims across documents by concept, not by string similarity.

Invariant: source_text must never be null. It is the citation anchor for every downstream discrepancy card.

2.4 Discrepancy

id · mismatch_type · severity · confidence · summary · explanation · claim_a_id · claim_b_id · created_at

claim_a_id always points to a real ExtractedClaim. claim_b_id may be null for CHECKLIST_GAP (one-sided: claim present in spec, absent from vendor). explanation is added after detection by the LLM explanation pass. mismatch_type is one of four values defined below.

3. Ingestion and Layout Parsing

3.1 Parser strategy per page

pdfplumber is the primary extraction engine. For each page, the parser runs text extraction and table extraction independently — tables are flattened to pipe-delimited text and appended to the page's text block. If the combined output is below a character threshold (80 chars), the page is rasterized at 200 DPI and passed to pytesseract for OCR.

Condition	Method	Confidence
Text-bearing PDF, clean layout	pdfplumber text	1.00
PDF with structured tables	pdfplumber tables + text	1.00
Sparse text (<80 chars)	pytesseract OCR	0.75
OCR unavailable or worse	pdfplumber best effort	0.50

3.2 Extraction method badge

The frontend displays the extraction method for every page and every claim. This makes messy-PDF handling visible to reviewers and explains lower-confidence flags from OCR pages. A claim from an OCR page is valid but should be verified against the original if contested.

4. Structured Extraction

The extractor is a two-pass hybrid. Pass 1 (regex) runs fast, deterministic patterns against page text. Pass 2 (LLM normalization) handles values regex could not fully classify. The two passes are combined with deduplication: for the same entity on the same page, the higher-confidence result wins.

4.1 Pass 1 — Regex extraction

Regex patterns are defined for each parameter type. Each match produces a claim with confidence 0.90–0.99. Regex runs first and is authoritative for the values it captures.

Claim Type	Patterns	Example
parameter	\d+\s?kW, \d+\s?kVA, \d+\s?Hz, \d+\s?V, \d+\s?RPM	1230 kW, 480 V, 60 Hz
assumption	standby prime continuous, indoor outdoor, auto.*start	Rating Type: CONTINUOUS
reference	UL 2200, UL 508A, NFPA 110, NEC, IEEE, IEC 61511, OSHA	per UL 2200
checklist	remote emergency stop, battery alarm, fault historian	remote emergency stop required

4.2 Pass 2 — LLM normalization

After regex, the LLM normalization pass runs on pages where additional structured values may remain: ambient temperature ranges, elevation limits, fuel consumption, enclosure IP/NEMA ratings, noise levels (dBA), warranty terms. The LLM receives the verbatim page text and a fixed Pydantic schema. It cannot introduce claim types or entity keys outside the defined taxonomy.

LLM constraint: the normalizer receives only source text from the current page.

It never receives information from other documents or from the comparison engine.

Its output is validated against a Pydantic schema before storage.

Confidence floor for LLM-only claims: 0.60.

4.3 Entity taxonomy

The entity field is a controlled vocabulary of normalized concept keys. The comparison engine groups claims by entity across documents — not by `parameter_name` string. This means 'standby generator set shall be rated standby power' and 'Rating Type: STANDBY' both map to entity `generator_rating_type`, and the comparison engine sees them as the same concept.

Entity keys are defined in `regex_rules.py` and the LLM normalization prompt. Neither the regex extractor nor the LLM may introduce entity keys outside this list. Adding a new entity type requires updating both files explicitly.

5. Comparison Engine

The comparison engine is fully deterministic. It runs four independent detectors in sequence. No LLM is involved at this stage. Each detector returns a list of raw flag dicts that include the source claims but not yet an explanation.

5.1 PARAMETER_MISMATCH — numeric diff

Groups claims by (entity, unit_normalized) across documents. For each entity present in two or more documents, compares numeric values. Flags if relative difference exceeds the threshold for that entity type.

```
severity = HIGH if diff > 10% | MEDIUM if diff > 3% | LOW otherwise  
confidence = min(conf_a, conf_b) × (1 - diff × 0.5), floored at 0.30
```

Example: voltage entity, `doc_a = 480V`, `doc_b = 600V` → `rel_diff = 25%` → severity HIGH, confidence ~0.71.

5.2 ASSUMPTION_MISMATCH — enum conflict

Enum parameters are categorical with a defined set of valid values: `generator_rating_type = {STANDBY, PRIME, CONTINUOUS}`, `frequency = {50HZ, 60HZ}`, `operation_mode = {AUTOMATIC, MANUAL, BOTH}`. If the same entity has different normalized values across two documents, a conflict is flagged.

Flagship example: Lakeland spec → `generator_rating_type = STANDBY`

Caterpillar 3512B datasheet → generator_rating_type = CONTINUOUS
→ ASSUMPTION_MISMATCH, severity HIGH, confidence 0.92
This is a real procurement-blocking discrepancy in the demo documents.

5.3 REFERENCE_GAP — standards set arithmetic

Collects the set of standards references from all owner_spec and control_spec documents (the required set). For each vendor_submittal document, computes the difference: required \ present. Each missing standard generates a REFERENCE_GAP flag pointing to the claim that requires it and noting its absence from the vendor document.

```
required = union(refs from owner_spec docs, refs from control_spec docs)
```

```
gaps = required - refs_in_vendor_submittal
```

→ one REFERENCE_GAP flag per missing standard

5.4 CHECKLIST_GAP — required item presence

Checklist items are extracted from spec documents using regex term lists (e.g. remote emergency stop → search terms: 'remote emergency stop', 'remote stop', 'emergency stop...remote'). The presence check scans the full text of vendor submittal documents for each required item. Absence generates a CHECKLIST_GAP flag citing the page where the requirement was stated in the spec.

Required Item	Search Terms (sample)	Severity if Absent
Remote emergency stop	remote emergency stop · remote stop · emergency stop remote	HIGH
Auto-start on power loss	automatic start · auto-start · autostart · automatic transfer	HIGH
Overcurrent protection	overcurrent protection · breaker · circuit breaker	HIGH
Battery low alarm	battery low · battery alarm · low battery	MEDIUM
Fault/alarm historian	fault historian · alarm log · alarm historian · event log	MEDIUM
Load testing requirement	load test · acceptance test · final acceptance · factory test	MEDIUM

6. Citation System

Citation traceability is the primary depth area of this implementation. Every flag shown in the UI must be traceable to its exact source — document, page number, verbatim snippet, and extraction method. This is what differentiates the system from a thin LLM wrapper.

6.1 Citation chain

The chain from flag to source is: `Discrepancy.claim_a_id` → `ExtractedClaim.source_text + page_number + document_id` → `Document.filename`. Each link is a database foreign key. There are no broken links permitted — the comparison engine enforces that every discrepancy has a valid `claim_a_id` before writing to the database.

6.2 Citation display format

Each discrepancy card in the UI shows the following structure:

Flag: Rating Type Mismatch · Severity: HIGH · Confidence: 0.92
Document A says:
"the generator set shall be rated standby power per NFPA 110..."
Source: lakeland_generator_spec.pdf, page 3 · extracted via regex
Document B says:
"Rating Type: CONTINUOUS"
Source: caterpillar_3512b_spec.pdf, page 1 · extracted via regex
Review note:
The project appears to require a standby-rated generator. The vendor submittal describes a continuous-rated unit. This should be verified before procurement.

6.3 One-sided citations (CHECKLIST_GAP)

Checklist gap flags cite only the document where the requirement was stated (`claim_a`). The absence side does not have a citation because there is no text to cite — the item is missing. The flag explanation makes this explicit: 'This item is required by [spec doc, page N] but was not found in [vendor doc].'

7. Confidence Scoring

Confidence is a computed value, not a model output. It is derived from three factors:

`confidence = (extraction_conf_a × 0.4) + (extraction_conf_b × 0.4) + (match_quality × 0.2)`

Factor	Value	Rationale
Regex match	0.90–0.99	Exact pattern match; unit explicitly present
LLM normalization (clean page)	0.70–0.80	LLM given clean pdfplumber text
LLM normalization (OCR page)	0.50–0.65	OCR introduces character errors
Numeric mismatch (exact diff)	1.0 × extraction	Two concrete values, direct comparison
Enum conflict (exact match)	1.0 × extraction	Both values in controlled vocabulary
Reference gap	0.85 × source conf	One-sided; absence is inferred
Checklist gap	0.80	Conservative; term matching is approximate

Confidence is displayed to reviewers alongside every flag. A LOW-confidence flag (< 0.65) is surfaced but visually de-emphasized. The reviewer is told the confidence score and its basis (extraction method) so they can calibrate trust accordingly.

8. LLM Explanation Pass

Claude is invoked exactly once per discrepancy, after it has been detected and stored. The prompt provides the two source snippets verbatim and the mismatch type. Claude is not told what the 'correct' value is. It is not asked to make a compliance determination. It is asked to write 2–3 sentences of cautious review language explaining why an engineer should look at this before proceeding.

System prompt constraint: 'Never say which document is correct. Never make a compliance judgment or certification claim. Use cautious language: appears to, may require, should be verified. Do not recommend a specific fix — recommend review. Keep under 60 words.'
--

If the LLM call fails, a fallback explanation is generated from a template using the mismatch type and parameter name. The flag is still displayed with all citations intact. The explanation is supplementary — its absence does not invalidate a flag.

9. Evaluation Design

A small golden set of manually labeled discrepancies is used to verify the comparison engine before demo. The golden set is derived from the three real demo PDFs (Lakeland owner spec, Casper College control spec, Caterpillar 3512B vendor datasheet).

9.1 Golden set

Expected Flag	Type	Expected Severity
Standby requirement vs. continuous-rated vendor sheet	ASSUMPTION_MISMATCH	HIGH
UL 2200 required in spec, not cited in vendor sheet	REFERENCE_GAP	MEDIUM
Remote emergency stop required, absent from vendor	CHECKLIST_GAP	HIGH
Battery low alarm required, absent from vendor	CHECKLIST_GAP	MEDIUM
NFPA 110 in spec, absent from vendor datasheet	REFERENCE_GAP	MEDIUM
Fault/alarm historian required, absent from vendor	CHECKLIST_GAP	MEDIUM

9.2 Metrics

- Claim extraction precision: fraction of extracted claims that are correct (verified against source PDF manually).
- Discrepancy precision: fraction of flagged discrepancies that correspond to real discrepancies in the golden set.
- Citation correctness: fraction of flags where the cited page and snippet match the actual source text.
- False positive rate: flags generated that are not in the golden set and cannot be verified in source documents.

A false positive rate above 20% on the golden set indicates a regex pattern is too broad or the LLM normalizer is classifying edge cases incorrectly. The fix is always in the extractor, not the comparison engine.

10. Technology Stack

Layer	Technology	Role
API	FastAPI + uvicorn	REST endpoints for upload, analysis, audit export
Database	SQLite + SQLAlchemy	Document, PageEvidence, ExtractedClaim, Discrepancy tables
PDF parsing	pdfplumber	Text and table extraction
OCR fallback	pytesseract + Pillow	Scanned / image-heavy pages
Extraction	Regex + Anthropic SDK	Regex first; LLM for normalization only
Comparison	Python (pure)	Deterministic; no LLM
Explanation	Anthropic SDK	claude-sonnet-4-20250514; cautious language only
Frontend	Streamlit	4-page review console
Audit report	ReportLab	PDF export; JSON always available
Validation	Pydantic v2	All LLM outputs validated before storage

11. SIS Design Analogy

Safety-instrumented systems are built around a principle of traceability from hazard to safeguard: sensor detects → logic solver decides → final control element acts. Every link in that chain is documented, tested, and auditable. A credible AI reviewer should apply the same principle to document review.

SIS Component	AI Reviewer Analog	Design Requirement
Sensor	Regex / OCR extractor	Must not miss high-value signals; high recall
Logic solver	Comparison engine	Must be deterministic and auditable; no LLM
Final control element	Review flag + citation	Must be actionable and traceable to evidence
Proof test	Golden set evaluation	Periodic verification against known discrepancies
Audit trail	Discrepancy + claim store	Immutable record of what was checked and found

A system that uses an LLM as its logic solver — where the LLM both detects and explains discrepancies — is equivalent to a safety system where the logic solver is a black box that cannot be proof-tested. InterLock AI deliberately separates detection (deterministic, inspectable) from explanation (LLM, bounded, post-hoc). This is the architectural decision that makes the system trustworthy in safety-critical review workflows.

This demo project does not replace engineering judgment. It reduces the probability of missed inconsistencies and makes reviews auditable.